

AlphaTrace = Solana trader PNL leaderboard + wallet profile + smart-money movement intelligence using Birdeye.

Board structure

Use these ticket categories:

Category	Meaning
FOUNDATIONS	repo, env, architecture contracts
BIRDEYE	API client and endpoint integration
BACKEND	application services and API routes
SCORING	Alpha Score and signal score logic
CACHE	database/cache/snapshots
FRONTEND	pages and UI components
DATA	schema and persistence
JOB	background refresh tasks
QA	validation, error handling, test wallets
DOCS	README, architecture docs, submission assets
DEMO	video, X post, judge-facing polish

Now we create tickets progressively.

Milestone 0 — Project foundation

This milestone creates the skeleton. No serious product logic yet.

Ticket 0.1 — Initialize AlphaTrace repo

Category: FOUNDATION

Goal: Create the project repository and basic app shell.

Tasks:

Task	Detail
Create app	Next.js + TypeScript
Add styling	Tailwind
Add linting	ESLint/Prettier if time allows
Add env file	<code>.env.example</code>
Add README placeholder	Explain project in 3–5 lines
Add folder structure	<code>lib/</code> , <code>components/</code> , <code>app/api/</code> , <code>docs/</code>

Acceptance criteria:

- App runs locally.
- Homepage loads.
- Repo has clean structure.
- `.env.example` lists required variables.

Ticket 0.2 — Define environment config

Category: FOUNDATION

Goal: Centralize all environment variables.

Env variables:

Variable	Purpose
<code>BIRDEYE_API_KEY</code>	Birdeye key

BIRDEYE_BASE_URL	API base URL
DEFAULT_CHAIN	likely solana
DATABASE_URL	later database connection
ADMIN_REFRESH_SEC RET	for protected refresh routes

Acceptance criteria:

- App fails clearly if required env vars are missing.
 - No API keys are hardcoded.
 - Backend imports config from one place only.
-

Ticket 0.3 — Define standard API response shape

Category: FOUNDATION

Goal: Every backend route returns a consistent shape.

Response shape:

Field	Meaning
ok	success/failure
data	payload
warnings	partial failures
freshness	fresh/cached/stale
generated At	timestamp
source	Birdeye-derived / AlphaTrace-derived
error	only on failure

Acceptance criteria:

- One shared response helper exists.
 - `/api/health` uses this shape.
 - Future routes do not invent their own response format.
-

Ticket 0.4 — Create `/api/health`

Category: BACKEND

Goal: Basic backend health endpoint.

Route:

GET `/api/health`

Returns:

Field	Purpose
app status	confirms server works
timestamp	confirms live response
env loaded	confirms config is present
version/name	identifies AlphaTrace

Acceptance criteria:

- Visiting `/api/health` returns `ok: true`.
 - No Birdeye call yet.
 - No database dependency yet.
-

Milestone 1 — Birdeye integration foundation

This is the first serious milestone. Nothing else matters if this is weak.

Ticket 1.1 — Build generic Birdeye HTTP client

Category: BIRDEYE

Goal: Create a single reusable client for Birdeye calls.

File:

`lib/birdeye/client.ts`

Responsibilities:

Responsibility	Detail
API key injection	Add required headers
Base URL handling	One base URL
Query params	clean construction
Timeout	prevent hanging requests
Error normalization	consistent errors
Logging	endpoint, status, duration
Chain handling	default to Solana

Acceptance criteria:

- No raw `fetch` calls outside `lib/birdeye`.
- Failed Birdeye calls return normalized errors.
- Successful calls return typed/normalized data.

Ticket 1.2 — Add Birdeye endpoint wrapper: wallet PNL summary

Category: BIRDEYE

Goal: Implement wallet PNL summary call.

Method:

```
getWalletPnlSummary(wallet, window)
```

Birdeye endpoint:

```
/wallet/v2/pnl/summary
```

Acceptance criteria:

- Takes wallet address and time window.
 - Returns normalized PNL summary.
 - Handles wallet with missing data.
 - Logs request duration.
-

Ticket 1.3 — Add Birdeye endpoint wrapper: wallet PNL details

Category: BIRDEYE

Goal: Implement token-level wallet PNL details.

Method:

```
getWalletPnlDetails(wallet, window)
```

Birdeye endpoint:

```
/wallet/v2/pnl/details
```

Acceptance criteria:

- Returns per-token PNL rows.
 - Sortable by profit/loss.
 - Handles empty result safely.
 - Does not crash if token metadata is missing.
-

Ticket 1.4 — Add debug route for wallet PNL

Category: BACKEND

Goal: Test Birdeye wallet PNL integration from the browser/curl.

Route:

`GET /api/debug/wallet-pnl?wallet=<address>&window=7d`

Acceptance criteria:

- Calls PNL summary and details.
 - Returns standard API response.
 - Shows warnings if one endpoint fails.
 - Useful for backend debugging.
-

Ticket 1.5 — Add wallet address validation

Category: BACKEND

Goal: Reject invalid wallet addresses before hitting Birdeye.

Acceptance criteria:

- Empty wallet returns 400.
 - Obviously invalid wallet returns 400.
 - Valid-looking Solana address proceeds.
 - Error message is understandable.
-

Milestone 2 — Single wallet profile backend

This is the first actual AlphaTrace feature.

Ticket 2.1 — Add wallet net worth endpoint wrapper

Category: BIRDEYE

Method:

`getWalletNetWorth(wallet, window)`

Endpoint:

`/wallet/v2/net-worth`

Acceptance criteria:

- Returns time-series data.
 - Handles empty history.
 - Normalizes timestamps and values.
-

Ticket 2.2 — Add wallet token list endpoint wrapper

Category: BIRDEYE

Method:

`getWalletTokenList(wallet)`

Endpoint:

`/v1/wallet/token_list`

Acceptance criteria:

- Returns current holdings.
 - Includes token address, balance, value if available.
 - Handles dust tokens reasonably.
-

Ticket 2.3 — Add first-funded endpoint wrapper

Category: BIRDEYE

Method:

`getWalletFirstFunded(wallet)`

Endpoint:

`/wallet/v2/tx/first-funded`

Acceptance criteria:

- Returns first funded date if available.
 - Converts to wallet age in days.
 - Missing first-funded data becomes a warning, not a crash.
-

Ticket 2.4 — Add token metadata wrapper

Category: BIRDEYE

Method:

```
getTokenMetadata(tokens[])
```

Endpoint:

```
/defi/v3/token/meta-data/multiple
```

Acceptance criteria:

- Takes many token addresses.
 - Returns symbol, name, logo, decimals where available.
 - Handles unknown tokens.
-

Ticket 2.5 — Build WalletProfileService

Category: BACKEND

Goal: Compose all wallet-related Birdeye data into one internal profile.

File:

```
lib/services/walletProfileService.ts
```

Inputs:

Input	Meaning
wallet address	target wallet
window	24h / 7d / 30d

Composes:

Data	Source
PNL summary	PNL summary endpoint
Token PNL details	PNL details endpoint
Net worth history	net worth endpoint
Current holdings	token list endpoint
Wallet age	first-funded endpoint
Token labels	metadata endpoint

Acceptance criteria:

- Returns a single `WalletProfile` object.
 - If one data source fails, the rest still returns.
 - Warnings list failed/missing pieces.
 - No frontend-specific formatting inside the service.
-

Ticket 2.6 — Create wallet profile API route

Category: `BACKEND`

Route:

`GET /api/wallets/:address/profile?window=7d`

Acceptance criteria:

- Validates wallet.
 - Calls `WalletProfileService`.
 - Returns standard API response.
 - Works via curl.
-

Milestone 3 — Single wallet profile frontend

Now you build the first visible product.

Ticket 3.1 — Build wallet search input

Category: FRONTEND

Component:

WalletSearchInput

Behavior:

Behavior	Detail
Paste wallet	user enters address
Validate minimally	required input
Submit	redirects to wallet page
Loading state	visible
Error state	visible

Acceptance criteria:

- User can paste address.
- Submitting navigates to `/wallet/[address]`.
- Invalid/empty input is handled.

Ticket 3.2 — Create wallet profile page

Category: FRONTEND

Page:

`/wallet/[address]`

Acceptance criteria:

- Calls wallet profile API.
 - Shows loading state.
 - Shows error state.
 - Shows wallet address.
 - Shows generated timestamp/freshness.
-

Ticket 3.3 — Build wallet summary cards

Category: `FRONTEND`

Component:

`WalletSummaryCards`

Cards:

Card	Meaning
Total PNL	profit/loss
ROI	efficiency
Wallet age	anti-luck signal
Current net worth	portfolio value if available

Acceptance criteria:

- Values are formatted cleanly.
 - Missing values show “Unavailable,” not broken UI.
 - Negative PNL is visually distinct.
-

Ticket 3.4 — Build token PNL table

Category: `FRONTEND`

Component:

TokenPnlTable

Columns:

Column	Meaning
Token	name/symbol
Realized PNL	token profit
ROI	token efficiency
Buys/Sells	activity if available
Last activity	recency if available

Acceptance criteria:

- Shows top winners and losers.
 - Sorts by PNL.
 - Handles missing token metadata.
-

Ticket 3.5 — Build net worth chart

Category: FRONTEND

Component:

NetWorthChart

Acceptance criteria:

- Shows wallet net worth over time.
 - Handles no chart data.
 - Does not block rest of page if unavailable.
-

Ticket 3.6 — Build holdings table

Category: FRONTEND

Component:

HoldingsTable

Columns:

Column	Meaning
Token	held token
Balance	quantity
Value USD	current value
Portfolio weight	concentration

Acceptance criteria:

- Shows current wallet exposure.
 - Filters tiny dust if needed.
 - Missing values do not break UI.
-

Milestone 4 — Wallet Alpha Score

Now the product becomes AlphaTrace, not just a Birdeye viewer.

Ticket 4.1 — Define wallet score input model

Category: SCORING

Goal: Decide exactly what data enters the scoring function.

Input model:

Field	Source
realized PNL	PNL summary
ROI	PNL summary
win rate	PNL summary/details if available
token PNL distribution	PNL details

wallet age	first-funded
recent activity	PNL/details/holdings
trade count	PNL data if available

Acceptance criteria:

- Score function has a typed input.
- Missing values are allowed.
- Missing values reduce confidence, not necessarily score.

Ticket 4.2 — Implement Alpha Score formula

Category: SCORING

File:

`lib/scoring/walletAlphaScore.ts`

Formula:

Component	Weight
Realized PNL	25%
ROI	20%
Win rate	15%
Consistency	15%
Wallet age	10%
Recency/activity	10%
Risk/data penalty	-5%

Acceptance criteria:

- Returns score from 0–100.
- Same input always returns same score.
- Score is clamped between 0 and 100.
- Breakdown is returned.

Ticket 4.3 — Implement confidence score

Category: **SCORING**

Goal: Avoid overranking wallets with weak data.

Confidence factors:

Factor	Reason
trade count	more trades = stronger evidence
number of tokens traded	avoids one-token luck
wallet age	old wallet = more credible
data completeness	missing data lowers confidence
volume	tiny wallet may be noisy

Acceptance criteria:

- Returns **high**, **medium**, or **low**.
- Explains why confidence is low.
- Wallet with one lucky token should not look unquestionable.

Ticket 4.4 — Implement wallet archetypes

Category: **SCORING**

Archetypes:

Archetype	Meaning
Elite Alpha	high score, high confidence
Strong Trader	good performance
Alpha Hunter	high ROI, aggressive upside

Consistent Compounder	steady, reliable performance
One-Hit Wonder	one token dominates profits
Rotator	many token rotations
Inactive Winner	good history, low recent activity
Exit Liquidity	poor performance

Acceptance criteria:

- Every analyzed wallet receives one archetype.
 - Archetype is rule-based.
 - Archetype has explanation text.
-

Ticket 4.5 — Attach Alpha Score to wallet profile

Category: BACKEND

Goal: Wallet profile API includes score output.

Acceptance criteria:

- `/api/wallets/:address/profile` returns `alphaScore`.
 - Includes score, class, confidence, breakdown, flags, archetype.
 - Existing frontend still works.
-

Ticket 4.6 — Build Alpha Score UI

Category: FRONTEND

Components:

Component	Purpose
<code>AlphaScoreBadge</code>	score/class

<code>AlphaScoreBreakdown</code>	component explanations
<code>ConfidenceBadge</code>	data reliability
<code>ArchetypePanel</code>	memorable wallet type

Acceptance criteria:

- Wallet page clearly explains why the wallet scored that way.
 - Score is not just a number.
 - Confidence is visible.
-

Milestone 5 — Database and cache foundation

This comes after score/profile shape is stable.

Ticket 5.1 — Choose and configure database

Category: `DATA`

Recommended: Supabase Postgres + Prisma, or plain Postgres.

Acceptance criteria:

- Database connection works.
 - Migration tool works.
 - App can read/write one test record.
-

Ticket 5.2 — Create wallet tables

Category: `DATA`

Tables:

`wallets`

Field	Purpose
address	primary wallet ID
first_seen_at	AlphaTrace discovery time
first_funded_at	wallet age
last_refreshed_at	cache freshness

wallet_profile_snapshots

Field	Purpose
wallet_address	target wallet
window	time window
summary_json	normalized summary
pnl_details_json	token PNL
holdings_json	portfolio
net_worth_json	chart
alpha_score_json	score
generated_at	snapshot time

Acceptance criteria:

- Migrations run successfully.
- Wallet profile can be saved.
- Wallet profile can be read back.

Ticket 5.3 — Implement wallet profile cache repository

Category: **CACHE**

File:

`lib/repositories/walletProfileRepository.ts`

Methods:

Method	Purpose
<code>getFreshSnapshot(wallet, window)</code>	read cache
<code>saveSnapshot(profile)</code>	write cache
<code>getLatestSnapshot(wallet, window)</code>	stale fallback

Acceptance criteria:

- Cache lookup works.
 - Cache save works.
 - Stale fallback works.
-

Ticket 5.4 — Add TTL rules

Category: `CACHE`

File:

`lib/cache/ttl.ts`

TTL values:

Data	TTL
wallet profile	10–30 min
holdings	2–5 min
net worth	30–60 min
token metadata	24h
first-funded	long/permanent

Acceptance criteria:

- TTLs are defined centrally.
 - Services do not hardcode TTL values.
-

Ticket 5.5 — Wire cache into WalletProfileService

Category: CACHE

Behavior:

1. check fresh snapshot
2. return if fresh
3. otherwise fetch Birdeye
4. save new snapshot
5. if fetch fails, return stale snapshot with warning

Acceptance criteria:

- Repeated wallet page loads use cache.
 - Stale data can be returned safely.
 - Response says whether data is fresh/cached/stale.
-

Milestone 6 — Leaderboard backend

Now we build the public ranking.

Ticket 6.1 — Add trader gainers/losers endpoint wrapper

Category: BIRDEYE

Method:

`getTraderGainersLosers(window)`

Endpoint:

`/trader/gainers-losers`

Acceptance criteria:

- Returns candidate wallet addresses.
 - Handles pagination/limits if available.
 - Normalizes wallet rows.
-

Ticket 6.2 — Define leaderboard entry model

Category: **BACKEND**

Model fields:

Field	Meaning
rank	final AlphaTrace rank
wallet	address
pnlUsd	raw profit
roiPercent	efficiency
alphaScore	derived score
confidence	reliability
walletAgeDays	anti-luck
archetype	wallet type

Acceptance criteria:

- Model is typed.
 - Frontend-ready but not frontend-specific.
-

Ticket 6.3 — Build LeaderboardService

Category: **BACKEND**

Flow:

1. fetch candidate wallets

2. deduplicate
3. enrich each with PNL summary
4. fetch first-funded data
5. compute Alpha Score
6. sort by score
7. return ranked entries

Acceptance criteria:

- Produces ranked wallet list.
 - Skips wallets that cannot be analyzed.
 - Includes warnings for skipped wallets.
-

Ticket 6.4 — Create leaderboard database tables

Category: DATA

Tables:

leaderboard_snapshots

Field	Purpose
id	snapshot
window	24h/7d/30d
generated_at	timestamp
wallet_count	analyzed count

leaderboard_entries

Field	Purpose
snapshot_id	parent
wallet_addresses	wallet
rank	rank

pnl_usd	profit
roi_percent	ROI
alpha_score	score
confidence	confidence
archetype	type

Acceptance criteria:

- Leaderboard snapshot can be persisted.
 - Latest leaderboard can be read from DB.
-

Ticket 6.5 — Create leaderboard API route

Category: **BACKEND**

Route:

`GET /api/leaderboard?window=7d&limit=50`

Acceptance criteria:

- Returns latest leaderboard.
 - Generates one if none exists.
 - Supports limit parameter.
 - Uses standard response shape.
-

Milestone 7 — Leaderboard frontend

Now the product starts looking real.

Ticket 7.1 — Build leaderboard page

Category: **FRONTEND**

Page:

[/leaderboard](#) or homepage /

Acceptance criteria:

- Fetches leaderboard API.
 - Displays generated timestamp.
 - Handles loading/error/empty states.
-

Ticket 7.2 — Build leaderboard table

Category: [FRONTEND](#)

Columns:

Column	Meaning
Rank	AlphaTrace rank
Wallet	address
PNL	profit
ROI	efficiency
Alpha Score	quality
Confidence	reliability
Wallet Age	credibility
Archetype	behavior class

Acceptance criteria:

- Table is readable.
 - Wallet rows link to profile page.
 - Scores are visually clear.
-

Ticket 7.3 — Add time-window selector

Category: [FRONTEND](#)

Windows:

- 24h
- 7d
- 30d

Acceptance criteria:

- Switching window refetches leaderboard.
 - Current window is visible.
 - Invalid window is rejected.
-

Ticket 7.4 — Add leaderboard empty/error states

Category: **FRONTEND**

Acceptance criteria:

- If no leaderboard exists, show clear message.
 - If Birdeye fails, show stale data if available.
 - User is not left with a blank page.
-

Milestone 8 — Search your wallet + trader card

This is the viral surface.

Ticket 8.1 — Build wallet analyze API route

Category: **BACKEND**

Route:

GET /api/wallets/:address/analyze?window=30d

Response:

Field	Meaning
wallet	address
pnlUsd	profit
roiPercent	ROI
alphaScore	score
confidence	reliability
archetype	type
percentile	compared to tracked wallets
shareText	X-ready summary

Acceptance criteria:

- Works for wallets not in leaderboard.
 - Does not claim global rank unless known.
 - Uses existing WalletProfileService.
-

Ticket 8.2 — Implement percentile comparison

Category: SCORING

Goal: Compare searched wallet against current tracked sample.

Acceptance criteria:

- Uses latest leaderboard entries.
 - Returns approximate percentile.
 - If no leaderboard exists, percentile is omitted.
-

Ticket 8.3 — Build trader card component

Category: FRONTEND

Component:

TraderCard

Shows:

Field	Meaning
wallet	shortened address
Alpha Score	main result
archetype	identity
PNL	performance
percentile	comparison
confidence	reliability

Acceptance criteria:

- Looks shareable.
 - Does not overcrowd information.
 - Works on mobile.
-

Ticket 8.4 — Add Share to X button

Category: FRONTEND

Goal: Help community score.

Acceptance criteria:

- Generates X share URL.
 - Includes AlphaTrace name.
 - Includes wallet score/archetype.
 - Does not expose sensitive/private data beyond public wallet address.
-

Milestone 9 — Smart-money feed backend

This is the second core product pillar.

Ticket 9.1 — Define smart-money candidate wallet source

Category: **BACKEND**

Rule:

Use latest leaderboard top N wallets.

Example:

Parameter	Value
top wallets	25–100
minimum Alpha Score	60
minimum confidence	medium
window	7d/30d

Acceptance criteria:

- Smart-money feed does not use random wallets.
 - Candidate wallets come from scored leaderboard.
 - Filters are explicit.
-

Ticket 9.2 — Build smart wallet holdings aggregator

Category: **BACKEND**

Goal: Gather tokens held by top wallets.

Flow:

1. load top wallets
2. fetch or read cached holdings
3. remove dust holdings
4. group by token address

5. attach participating wallets

Acceptance criteria:

- Produces token → wallets mapping.
 - Ignores tiny dust positions.
 - Uses cache where possible.
-

Ticket 9.3 — Add token market data wrapper

Category: BIRDEYE

Method:

`getTokenMarketData(token)`

Endpoint:

`/defi/v3/token/market-data`

Acceptance criteria:

- Returns liquidity/volume/price if available.
 - Handles missing market data.
-

Ticket 9.4 — Add token security wrapper

Category: BIRDEYE

Method:

`getTokenSecurity(token)`

Endpoint:

`/defi/token_security`

Acceptance criteria:

- Returns basic security fields.
- Missing data becomes warning.

- Critical security issue can penalize signal.
-

Ticket 9.5 — Define token signal model

Category: **BACKEND**

Model fields:

Field	Meaning
token	token address
symbol/name/logo	metadata
smartWalletCount	number of top wallets involved
averageAlphaScore	quality of wallets
combinedWalletPnl	credibility
liquidityUsd	tradability
volume24h	market activity
securityStatus	basic risk
signalScore	final score
participants	wallets causing signal
explanation	plain-English reason

Acceptance criteria:

- Model is typed.
 - Participants are included.
 - Explanation is required.
-

Ticket 9.6 — Build SmartMoneyFeedService

Category: **BACKEND**

Flow:

1. load top scored wallets
2. aggregate holdings by token
3. enrich token metadata
4. enrich market data
5. enrich security data
6. compute signal score
7. sort signals
8. return top tokens

Acceptance criteria:

- Returns ranked token signals.
 - Every signal has participants.
 - Every signal has explanation.
 - Uses only AlphaTrace smart wallets, not global token scan.
-

Milestone 10 — Token signal scoring

Now make the smart-money feed interpretable.

Ticket 10.1 — Implement token signal score formula

Category: **SCORING**

Formula:

Component	Weight
smart wallet count	25%
average Alpha Score	20%
combined wallet PNL	20%
portfolio exposure	10%
liquidity	10%
security	10%

freshness 5%

Acceptance criteria:

- Returns score 0–100.
 - Same input gives same score.
 - Produces breakdown.
-

Ticket 10.2 — Implement signal labels

Category: **SCORING**

Labels:

Score	Label
80–100	Strong Smart Money Signal
60–79	Emerging Signal
40–59	Weak / Noisy Signal
0–39	Ignore

Acceptance criteria:

- Every token signal has a label.
 - Label is consistent with score.
-

Ticket 10.3 — Generate signal explanations

Category: **SCORING**

Goal: Convert score into user-facing explanation.

Example:

“5 high-alpha wallets currently hold this token. Their average Alpha Score is 81, with combined realized PNL of \$420k. Liquidity appears acceptable and no critical security warning was found.”

Acceptance criteria:

- Explanation mentions wallet count.
 - Explanation mentions wallet quality.
 - Explanation mentions security/liquidity when available.
 - No price prediction language.
-

Milestone 11 — Smart-money feed persistence/cache

Ticket 11.1 — Create token signal tables

Category: DATA

Tables:

token_signal_snapshots

Field	Purpose
id	snapshot
token_address	token
signal_score	score
signal_label	label
smart_wallet_count	number involved
average_alpha_score	wallet quality
combined_wallet_pnl	credibility
liquidity_usd	tradability
volume_24h	market activity
security_status	risk
generated_at	timestamp

token_signal_participants

Field	Purpose
signal_snapshot_id	parent signal
wallet_address	involved wallet
wallet_alpha_score	wallet quality
wallet_pnl_usd	wallet credibility
holding_value_usd	exposure
portfolio_weight	conviction proxy
reason	why included

Acceptance criteria:

- Signals can be saved.
- Participants can be saved.
- Latest signal feed can be loaded.

Ticket 11.2 — Add smart-money feed API route

Category: **BACKEND**

Route:

GET /api/smart-money/feed?window=7d&limit=30

Acceptance criteria:

- Returns latest signal snapshot.
 - Generates snapshot if none exists.
 - Uses standard response shape.
 - Shows freshness timestamp.
-

Ticket 11.3 — Add smart-money feed cache behavior

Category: **CACHE**

TTL:

5–15 minutes.

Acceptance criteria:

- Repeated feed loads do not recompute everything.
 - Stale feed can be returned if refresh fails.
 - Response marks cached/stale status.
-

Milestone 12 — Smart-money feed frontend

Ticket 12.1 — Build smart-money page

Category: **FRONTEND**

Page:

`/smart-money`

Acceptance criteria:

- Fetches feed API.
 - Shows loading/error/empty states.
 - Displays generated timestamp.
-

Ticket 12.2 — Build token signal card

Category: **FRONTEND**

Component:

TokenSignalCard

Shows:

Field	Meaning
token symbol/name	identity
signal score	strength
label	interpretation
smart wallet count	adoption by profitable wallets
average Alpha Score	wallet quality
combined wallet PNL	credibility
liquidity/volume	market context
security badge	basic safety
explanation	why shown

Acceptance criteria:

- User can understand signal without reading docs.
 - Explanation is visible.
 - No token card looks like financial advice.
-

Ticket 12.3 — Build participants drawer/modal

Category: FRONTEND

Goal: Show why token was included.

Shows:

Field	Meaning
wallet	involved wallet
wallet Alpha Score	quality

wallet PNL	credibility
holding value	exposure
portfolio weight	conviction proxy

Acceptance criteria:

- User can audit the signal.
 - Wallets link to wallet profile pages.
-

Milestone 13 — Background refresh and admin controls

Ticket 13.1 — Add protected admin refresh route for leaderboard

Category: JOBS

Route:

POST /api/admin/refresh-leaderboard

Acceptance criteria:

- Requires admin secret.
 - Rebuilds leaderboard snapshot.
 - Returns count of wallets analyzed.
 - Logs failures.
-

Ticket 13.2 — Add protected admin refresh route for smart-money feed

Category: JOBS

Route:

POST /api/admin/refresh-smart-money

Acceptance criteria:

- Requires admin secret.
 - Rebuilds smart-money feed.
 - Returns number of token signals generated.
 - Uses latest leaderboard.
-

Ticket 13.3 — Add scheduled refresh

Category: JOBS

Frequency:

Job	Frequency
leaderboard refresh	15–30 min
smart-money refresh	10–15 min
token metadata refresh	lazy/daily

Acceptance criteria:

- Scheduled job works in deployed environment.
 - Refresh does not expose secret.
 - Failed jobs are logged.
-

Milestone 14 — Reliability and QA

Ticket 14.1 — Create test wallet list

Category: QA

Goal: Maintain a small list of wallets for testing.

Types:

Wallet type	Purpose
profitable wallet	positive PNL
losing wallet	negative PNL
new wallet	low age
inactive wallet	stale behavior
empty wallet	edge case
invalid address	validation

Acceptance criteria:

- Test wallets documented.
 - Each core route tested with at least 3 wallet types.
-

Ticket 14.2 — Add partial failure handling

Category: QA

Goal: App survives missing endpoint data.

Cases:

Failure	Expected behavior
PNL details missing	still show summary
net worth missing	chart unavailable
metadata missing	show token address
security missing	warning
Birdeye timeout	stale cache if available

Acceptance criteria:

- No full-page crashes.
 - Warnings are returned and visible where useful.
-

Ticket 14.3 — Add API credit awareness

Category: QA

Goal: Avoid burning Birdeye credits blindly.

Tasks:

Task	Purpose
Log endpoint calls	see usage
Track cache hits	verify savings
Add debug counter	estimate cost
Avoid repeated refresh spam	protect API quota

Acceptance criteria:

- You can tell how many Birdeye calls a page causes.
 - Cached page loads cause fewer calls.
-

Milestone 15 — Presentation package

This is for judging.

Ticket 15.1 — Write README

Category: DOCS

Sections:

1. What AlphaTrace is
2. Problem

3. Features
4. Screenshots
5. Architecture
6. Birdeye endpoints used
7. Scoring model
8. Setup
9. Limitations
10. Future work

Acceptance criteria:

- A judge can understand the project without running it.
 - Endpoints used are clearly listed.
-

Ticket 15.2 — Write architecture doc

Category: **DOCS**

File:

`docs/architecture.md`

Must explain:

Section	Meaning
data flow	Birdeye → services → scores → UI
services	Leaderboard, WalletProfile, SmartMoneyFeed
cache	snapshot strategy
scoring	deterministic intelligence layer
limitations	what app does not claim

Acceptance criteria:

- Clear enough for technical judge.
 - No overclaiming.
-

Ticket 15.3 — Write scoring model doc

Category: DOCS

File:

`docs/scoring-model.md`

Must explain:

- Alpha Score formula
- confidence score
- wallet archetypes
- token signal score
- why scores are deterministic
- limitations

Acceptance criteria:

- Score is auditable.
 - Judges can see technical depth.
-

Ticket 15.4 — Create demo script

Category: DEMO

Demo flow:

1. Open leaderboard
2. Explain Alpha Score
3. Click a top wallet
4. Show wallet profile
5. Search a wallet
6. Show trader card
7. Open smart-money feed
8. Explain token signal participants
9. Mention Birdeye endpoints used

Acceptance criteria:

- Demo can be done in under 2 minutes.
- No rambling.
- Product value is obvious.

Ticket 15.5 — Create X launch post

Category: DEMO

Angle:

“We built AlphaTrace: a Solana intelligence dashboard that ranks profitable wallets by PNL and turns their activity into smart-money token signals.”

Must include:

- product link
- GitHub link
- demo screenshots/video
- tag Birdeye
- use required hashtag if competition requires it

The bounty context specifically cares about X engagement/community support, so this ticket is not optional.

Recommended ticket execution order

This is the order I would actually work in:

Order	Ticket
1	0.1 Initialize repo
2	0.2 Env config
3	0.3 API response shape
4	0.4 Health route
5	1.1 Birdeye client
6	1.2 PNL summary wrapper
7	1.3 PNL details wrapper

8	1.4 Debug wallet PNL route
9	1.5 Wallet validation
10	2.1 Net worth wrapper
11	2.2 Token list wrapper
12	2.3 First-funded wrapper
13	2.4 Token metadata wrapper
14	2.5 WalletProfileService
15	2.6 Wallet profile API
16	3.1 Wallet search input
17	3.2 Wallet profile page
18	3.3 Summary cards
19	3.4 Token PNL table
20	3.5 Net worth chart
21	3.6 Holdings table
22	4.1 Score input model
23	4.2 Alpha Score
24	4.3 Confidence score
25	4.4 Archetypes
26	4.5 Attach score to profile
27	4.6 Score UI
28	5.1 Database
29	5.2 Wallet tables
30	5.3 Cache repository
31	5.4 TTL rules
32	5.5 Wire wallet cache

33	6.1 Trader gainers/losers wrapper
34	6.2 Leaderboard entry model
35	6.3 LeaderboardService
36	6.4 Leaderboard tables
37	6.5 Leaderboard API
38	7.1 Leaderboard page
39	7.2 Leaderboard table
40	7.3 Window selector
41	7.4 Empty/error states
42	8.1 Wallet analyze API
43	8.2 Percentile comparison
44	8.3 Trader card
45	8.4 Share to X
46	9.1 Smart wallet candidate source
47	9.2 Holdings aggregator
48	9.3 Token market data wrapper
49	9.4 Token security wrapper
50	9.5 Token signal model
51	9.6 SmartMoneyFeedService
52	10.1 Token signal score
53	10.2 Signal labels
54	10.3 Signal explanations
55	11.1 Token signal tables
56	11.2 Smart-money feed API

57	11.3 Smart-money cache
58	12.1 Smart-money page
59	12.2 Token signal card
60	12.3 Participants drawer
61	13.1 Refresh leaderboard route
62	13.2 Refresh smart-money route
63	13.3 Scheduled refresh
64	14.1 Test wallet list
65	14.2 Partial failure handling
66	14.3 API credit awareness
67	15.1 README
68	15.2 Architecture doc
69	15.3 Scoring model doc
70	15.4 Demo script
71	15.5 X launch post

The first sprint

For the first working session, only do these:

Ticket	Owner	Outcome
0.1 Repo init	coder	app exists
0.2 Env config	coder	API key config
0.3 Response shape	architect + coder	standard API shape
0.4 Health route	coder	backend sanity check
1.1 Birdeye client	coder	integration boundary

1.2 PNL summary wrapper	coder	first real endpoint
1.3 PNL details wrapper	coder	token-level PNL
1.4 Debug wallet PNL route	coder	test via curl
1.5 Wallet validation	coder	safe input

Your first milestone is complete when this works:

```
GET /api/debug/wallet-pnl?wallet=<wallet>&window=7d
```

and returns clean normalized PNL data.

That is where you start.